

SISTEMAS MULTIDIMENSIONALES

Práctica 2: Integración de Sistemas (Diseño del Componente ETL)

Francisco Javier Santos Rivas

Índice

1. Introducción.....	3
2. Data profiling.....	3
2.1. Por qué es necesario conocer los datos en el contexto de AgroCeuta S.L.....	3
2.2. Anomalías detectadas - Resultados del data profiling.....	4
3. Diseño del Componente ETL.....	10
3.1 Integración de la dimensión meteorológica.....	10
3.2. Diseño del flujo ETL para AgroCeuta.....	11
3.2.1. Preparación para simular el proceso ETL.....	12
3.2.2 Diagrama de Arquitectura ETL y flujo de transformación.....	13
3.2.3 Justificaciones de diseño.....	15
4. Implementación del componente ETL con Pentaho.....	15
4.1 Catálogo de transformaciones.....	15
4.2 Descripción detallada de cada paso.....	16
5. Integración de objetivos en consulta analítica.....	19
5.1 Actualización del catálogo de transformaciones.....	19
5.2 Flujo de transformación propuesto.....	19
5.3. Resultado final de la transformación.....	22
Anexo A. Archivos adjuntos entregados.....	23

1. Introducción

En esta práctica continúo directamente donde finalizó la Práctica 1, en la que se diseñó e implementó el sistema transaccional (OLTP) de AgroCeuta, una empresa ficticia del sector agroalimentario dedicada a la exportación de productos desde la provincia de Ceuta. En aquella primera entrega se construyó: el modelo relacional con cinco departamentos (Producción, Calidad, Ventas, Logística y Finanzas), el modelo dimensional en estrella y snowflake, y una primera aproximación al componente ETL mediante scripts SQL.

En esta segunda práctica se aborda la automatización de ese proceso ETL utilizando Pentaho Data Integration (PDI), herramienta de integración de datos ampliamente utilizada en entornos de producción. Para ello, la práctica se separa en 4 fases:

1. Análisis de calidad de datos o data profiling sobre un dataset externo de observaciones meteorológicas
2. Diseño de la arquitectura ETL completa con tres capas (OLTP → Staging → DW)
3. Implementación de las transformaciones en Pentaho
4. Integración de un sistema externo y conexión con el modelo OLAP.

2. Data profiling

2.1. Por qué es necesario conocer los datos en el contexto de AgroCeuta S.L.

En un sistema productivo de inteligencia de negocio como el que se está construyendo para AgroCeuta S.L., la calidad de los datos determina directamente la calidad de las decisiones. AgroCeuta opera en un sector donde las condiciones climáticas tienen un impacto directo en la producción, la logística y las ventas:

- Una tormenta en el puerto de Algeciras puede retrasar un envío.
- Una ola de calor en el mes de julio puede anticipar la campaña de recolección.
- Unas precipitaciones por encima de la media pueden afectar al rendimiento por hectárea de una parcela determinada.

Para poder cruzar estas variables externas con los datos internos del OLTP (costes de lote, tiempos de envío, márgenes por campaña), es imprescindible que los datos meteorológicos que entren en el Data Warehouse sean coherentes, completos y libres de errores. Un valor de temperatura de -99 °C o un nombre de ciudad escrito de tres formas distintas no son solo problemas técnicos: son datos que romperían agregaciones, falsearían medias y contaminarían dimensiones del DW, haciendo inútiles los informes analíticos que dependen de ellos.

El data profiling es, por tanto, el primer paso obligatorio antes de cualquier transformación ETL. Consiste en examinar sistemáticamente los datos de la fuente para identificar su estructura, detectar anomalías y definir las reglas de limpieza que aplicará el proceso ETL. En esta práctica, el data profiling se realiza con SQL puro sobre el dataset meteorológico de observaciones diarias (18.282 registros, ciudades españolas, periodo 2020-2024), ejecutando las queries del script 02_profiling_queries.sql sobre el esquema meteo de la base de datos.

Las seis dimensiones de calidad que se evalúan son: completitud (campos nulos), unicidad (duplicados), consistencia (formatos de ciudad), exactitud (valores imposibles), validez (vocabulario controlado) e integridad referencial (claves foráneas huérfanas). Los resultados se recogen en la tabla de anomalías del apartado siguiente.

2.2. Anomalías detectadas - Resultados del data profiling

ID	Anomalía	Query	Registros afectados	Regla de limpieza
A1	Error de sensor (-99)	P03	15	temp = -99 → NULL
A2	Precipitación negativa	P04	8	precip < 0 → 0.0
A3	Humedad fuera de [0, 100]	P05	5	humedad_pct < 0 OR > 100 → NULL
A4	Ciudad inconsistente	P11*	138	Normalizar el valor mediante LOWER(TRIM(ciudad)) y

				mapear cada variante a su nombre normalizado
A5	Temp mínima > máxima	P06	7	Marcar registro con flag_temp_invertida = TRUE; conservando el dato.
A6	FK huérfana estacion_id	P12	20	Registrar en un log de errores, sin eliminar el registro.
A7	Fila completamente nula	P15	3	Eliminar el registro del staging limpio
A8	Duplicado (fecha, ciudad)	P08	12	Conservar el registro con menor observacion.id; eliminar los restantes.
A9	Vocabulario inválido	P07	9	Sustituir por 'desconocido'

A continuación, explico cómo he observado cada anomalía:

A1: Ejecuto la Query P03 y obtengo lo siguiente:

```
agroceuta_dw=# -- P03: Valores imposibles en temperaturas (¿qué es -99.0?)
SELECT
  COUNT(*) FILTER (WHERE temp_max = -99.0) AS temp_max_error_sensor,
  COUNT(*) FILTER (WHERE temp_max < -50) AS temp_max_extrema_frio,
  COUNT(*) FILTER (WHERE temp_max > 60) AS temp_max_extrema_calor,
  COUNT(*) FILTER (WHERE temp_min < -50) AS temp_min_extrema_frio,
  COUNT(*) FILTER (WHERE temp_min > 50) AS temp_min_extrema_calor
FROM observaciones_diarias;
 temp_max_error_sensor | temp_max_extrema_frio | temp_max_extrema_calor | temp_min_extrema_frio | temp_min_extrema_calor
-----+-----+-----+-----+-----
15 | 15 | 0 | 0 | 0
(1 row)
```

Vemos que la condición temp_max = -99.0 se cumple 15 veces, por lo tanto hay 15 registros erróneos. Se pueden corregir asignando NULL.

A2: Ejecuto la Query P04 y obtengo lo siguiente:

```
agroceuta_dw=# -- P04: Precipitación negativa (físicamente imposible)
SELECT observacion_id, fecha, ciudad, precipitacion_mm
FROM observaciones_diarias
WHERE precipitacion_mm < 0
ORDER BY precipitacion_mm;
 observacion_id | fecha | ciudad | precipitacion_mm
-----+-----+-----+-----
8696 | 2023-10-19 | Barcelona | -4.8
3372 | 2024-03-24 | Sevilla | -4.3
12576 | 2024-06-01 | Bilbao | -3.8
16561 | 2020-04-27 | Almería | -3.0
7952 | 2021-10-05 | Barcelona | -2.6
14541 | 2024-10-17 | Zaragoza | -1.7
15527 | 2022-06-29 | Murcia | -0.7
9019 | 2024-09-06 | Barcelona | -0.2
(8 rows)
```

Vemos que hay 8 observaciones (8 registros) con precipitación negativa. Una posible corrección sería poner a 0 todos los valores de precipitación que sean menores que 0.

A3: Ejecuto la Query P05 y obtengo lo siguiente:

```
agroceuta_dw=# -- P05: Humedad fuera de rango [0, 100]
SELECT observacion_id, fecha, ciudad, humedad_pct
FROM observaciones_diarias
WHERE humedad_pct < 0 OR humedad_pct > 100
ORDER BY humedad_pct DESC;
observacion_id | fecha      | ciudad  | humedad_pct
-----+-----+-----+-----
16412 | 2024-11-30 | Murcia  | 133
1309 | 2023-08-01 | Granada | 132
3864 | 2020-07-28 | Málaga  | 123
14908 | 2020-10-18 | Murcia  | 113
10376 | 2023-05-25 | Valencia | 103
(5 rows)
```

Vemos que hay 5 observaciones (5 registros) con humedades imposibles. Como corrección, se podrían fijar en “NULL” aquellos humedad_pct que sean inferiores a 0 o superiores a 100.

A4: Este es más complejo y se necesitan 2 queries para obtener los registros afectados, P10 y P11. Obtengo lo siguiente:

```
agroceuta_dw=# -- P10: Variantes de nombres de ciudad (¿cuántas "versiones" de cada ciudad hay?)
SELECT
  LOWER(TRIM(ciudad)) AS ciudad_normalizada,
  COUNT(DISTINCT ciudad) AS variantes_encontradas,
  STRING_AGG(DISTINCT ciudad, ' | ' ORDER BY ciudad) AS listado_variantes,
  COUNT(*) AS total_registros
FROM observaciones_diarias
GROUP BY LOWER(TRIM(ciudad))
ORDER BY variantes_encontradas DESC, total_registros DESC;

-- P11: Registros con nombre de ciudad sospechoso (no en lista esperada)
SELECT DISTINCT ciudad
FROM observaciones_diarias
WHERE ciudad NOT IN (
  'Granada','Sevilla','Málaga','Madrid','Barcelona',
  'Valencia','Bilbao','Zaragoza','Murcia','Almería'
)
ORDER BY ciudad;
ciudad_normalizada | variantes_encontradas | listado_variantes | total_registros
-----+-----+-----+-----
granada | 4 | granada | Granada | GRANADA | Granada | 1820
sevilla | 3 | sevilla | Sevilla | SEVILLA | 1821
barcelona | 3 | barcelona | Barcelona | BARCELONA | 1820
madrid | 3 | madrid | Madrid | MADRID | 1817
malaga | 3 | malaga | Malaga | MALAGA | 20
málaga | 2 | Málaga | Málaga | 1808
zaragoza | 1 | Zaragoza | 1829
almeria | 1 | Almería | 1828
murcia | 1 | Murcia | 1828
bilbao | 1 | Bilbao | 1827
valencia | 1 | Valencia | 1827
madrid | 1 | Madrid | 14
barcelona | 1 | Barcelona | 9
grañada | 1 | Grañada | 8
sevilla | 1 | Sevilla | 6
(15 rows)

ciudad
-----
barcelona
BARCELONA
Barcelona
granada
GRANADA
Granada
Grañada
madrid
MADRID
Madrid
malaga
Malaga
MALAGA
málaga
Sevilla
sevilla
SEVILLA
(17 rows)
```

Sin embargo, el número de registros afectados no se puede extraer de esta forma. P10 cuenta todos los registros (tanto para ciudades bien escritas como mal escritas), y P11 solo dan las ciudades erróneas. La solución para obtener un número más preciso de registros afectados sería modificar la query P11 de esta forma:

```
agroceuta_dw=# SELECT ciudad, COUNT(*) AS registros_afectados
FROM observaciones_diarias
WHERE ciudad NOT IN (
    'Granada','Sevilla','Málaga','Madrid','Barcelona',
    'Valencia','Bilbao','Zaragoza','Murcia','Almería'
)
GROUP BY ciudad
ORDER BY registros_afectados DESC;
 ciudad | registros_afectados
-----+-----
MADRID  | 14
Madrid  | 14
GRANADA | 12
madrid  | 11
Barcelona | 9
Málaga  | 9
barcelona | 8
Granada  | 8
Grañada  | 8
MALAGA  | 8
málaga  | 6
sevilla  | 6
granada  | 6
SEVILLA  | 6
Sevilla  | 6
BARCELONA | 4
malaga   | 3
(17 rows)
```

Así, obtenemos cuántos registros están asociados a ciudades mal escritas (o al menos no como se esperan en la lista esperada). Sumando todos los registros afectados, tenemos un total de 138 registros afectados. Una limpieza adecuada sería normalizar el valor mediante LOWER(TRIM(ciudad)) y mapear cada variante a su nombre normalizado.

A5: Ejecuto la query P06 y obtengo lo siguiente:

```
agroceuta_dw=# -- P06: Temperatura mínima mayor que la máxima (coherencia interna)
SELECT
    observacion_id, fecha, ciudad,
    temp_min, temp_max,
    temp_min - temp_max AS diferencia_invalida
FROM observaciones_diarias
WHERE temp_min > temp_max
    AND temp_max <> -99.0 -- Excluir errores de sensor ya detectados
ORDER BY diferencia_invalida DESC;
 observacion_id | fecha      | ciudad  | temp_min | temp_max | diferencia_invalida
-----+-----+-----+-----+-----+-----
3326 | 2024-02-07 | Sevilla | 17.2 | 0.9 | 16.3
1721 | 2024-09-16 | Granada | 35.7 | 19.6 | 16.1
10236 | 2023-01-05 | Valencia | 7.3 | -7.7 | 15.0
12922 | 2020-05-12 | Zaragoza | 18.2 | 4.6 | 13.6
17774 | 2023-08-23 | Almería | 35.8 | 22.5 | 13.3
2104 | 2020-10-03 | Sevilla | 27.2 | 14.5 | 12.7
11304 | 2020-12-07 | Bilbao | 12.2 | 0.5 | 11.7
(7 rows)
```

Tenemos 7 registros que no cumplen con las condiciones, y por tanto necesitan ser limpiados. El script 03_limpieza_staging.sql nos da una pista

de cómo hacerlo: crear un flag que nos marque si la temperatura está invertida y marcarlo como TRUE en los casos que se den. Esto en sí no corrige nada, pero sí puede ser útil en una hipotética revisión manual.

A6: La query P12 nos permite ver esta anomalía:

```
agroceuta_dw=# -- P12: Estaciones referenciadas en observaciones que NO existen en el catálogo
SELECT
    o.estacion_id,
    COUNT(*) AS observaciones_huerfanas
FROM observaciones_diarias o
LEFT JOIN estaciones_meteorologicas e ON o.estacion_id = e.estacion_id
WHERE e.estacion_id IS NULL
GROUP BY o.estacion_id
ORDER BY o.estacion_id;
```

estacion_id	observaciones_huerfanas
51	4
52	4
53	1
55	1
57	2
58	3
59	1
60	4

(8 rows)

Vemos que hay 8 estaciones huérfanas, que afectan a un total de 20 observaciones (resultado de sumar todas las observaciones_huerfanas). Una solución, si no queremos perder el dato, será registrar estas anomalías en un log de errores (staging.etl_errores en el script 03).

A7: La query P02 solo calcula porcentajes de nulos por columna, pero no nos sirve para ver las filas completamente vacías (por ejemplo, en una columna temp_max puede ser NULL pero precipitacion_nm no). Sin embargo, la query P15 sí cuenta con una sección que calcula esta fila nula:

```
agroceuta_dw=# SELECT 'A7 - Registros completamente vacíos',
    COUNT(*) FILTER (WHERE temp_max IS NULL AND temp_min IS NULL AND precipitacion_mm IS NULL)
FROM observaciones_diarias;
```

?column?	count
A7 - Registros completamente vacíos	3

Vemos que hay 3 registros que están completamente vacíos. Lo más ideal sería eliminar estos registros del staging limpio.

A8: La query P08 permite identificar los registros afectados por la anomalía:

```

agroceuta_dw=# -- P08: Registros duplicados (misma fecha y ciudad)
SELECT
    fecha,
    ciudad,
    COUNT(*) AS registros_por_dia
FROM observaciones_diarias
WHERE temp_max IS NOT NULL -- Excluir registros vacíos
GROUP BY fecha, ciudad
HAVING COUNT(*) > 1
ORDER BY registros_por_dia DESC, fecha;

```

fecha	ciudad	registros_por_dia
2020-06-11	Madrid	2
2020-08-05	Madrid	2
2021-09-08	Barcelona	2
2023-02-14	Almería	2
2023-03-24	Barcelona	2
2023-04-03	Zaragoza	2
2023-05-27	Madrid	2
2023-07-02	Murcia	2
2023-07-28	Málaga	2
2023-11-21	Zaragoza	2
2024-01-25	Madrid	2
2024-05-29	Granada	2

(12 rows)

Tenemos 12 pares (fecha, ciudad) con doble registro. Puede ser una buena solución conservar el registro con menor observacion.id, eliminando los demás registros.

A9: Con la query P07 podemos analizar la anomalía:

```

agroceuta_dw=# -- P07: Condición climática con valores inválidos
SELECT
    condicion_climatica,
    COUNT(*) AS ocurrencias
FROM observaciones_diarias
GROUP BY condicion_climatica
ORDER BY ocurrencias DESC;

```

condicion_climatica	ocurrencias
Lluvia ligera	2150
Viento fuerte	2100
Lluvia fuerte	2027
Niebla	2019
Nublado	2018
Parcialmente nublado	2012
Tormenta	2003
Nieve	1974
Despejado	1970
	3
N/A	1
--	1
DESCONOCIDO	1
null	1
error	1
999	1

(16 rows)

Vemos que después de “Despejado” están todas las anomalías: 3 son “ (vacías), y N/A, --, DESCONOCIDO, null, error y 999 aparecen 1 vez cada uno, dando un total de 9 registros afectados. Podemos sustituir todos estos valores por “Desconocido”. La query P15 también podría servir, pero no detecta las 3 condiciones climáticas “ (vacías).

3. Diseño del Componente ETL

3.1 Integración de la dimensión meteorológica

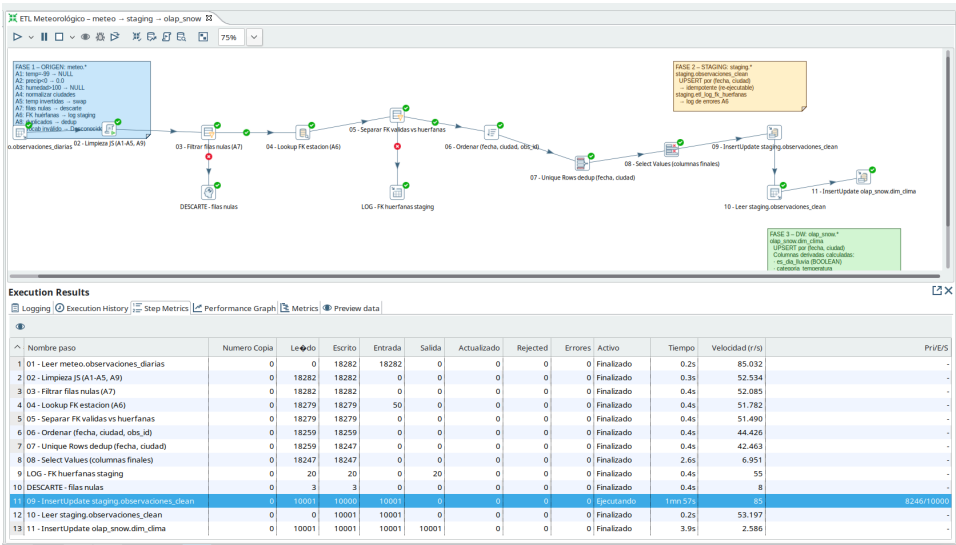
Arquitectura utilizada: El flujo usa una arquitectura de 3 capas (OLTP → Staging → DW). La capa de Staging actúa como zona de cuarentena y permite rollback seguro si el ETL falla. El flujo carga la dimension `olap_snow.dim_clima` con SCD Tipo 1.

Descripción del flujo (`limpieza_meteo_completo.ktr`):

Paso	Tipo	Función
01	Table Input	Extrae 18282 filas de <code>meteo.observaciones_diarias</code>
02	Limpieza JS (A1-A5, A9) → Modified JS	Corrige valores -99 → NULL; precio<0 → 0.0; humedad fuera de rango → NULL; ciudades inconsistentes, temp_min>temp_max invertidos, condición inválida → NULL
03	Filter Rows (A7)	Descarta filas completamente nulas
04	DB Lookup (A6)	Valida FK <code>estacion_id</code> contra <code>meteo.estaciones_meteorologicas</code> ; si no se cumple, <code>lkp_estacion</code> = -1.
05	Filter Rows (FK)	Bifurca el flujo: las FK válidas siguen, mientras que las huérfanas van a un log.
LOG	Insert/Update	Registra FKs inválidas en un log
06	Sort Rows	Ordenado por <code>fecha_id</code> , <code>ciudad_id</code> , <code>obs_id</code> en ASC.
07	Unique Rows	Elimina duplicados por clave (<code>fecha</code> , <code>ciudad</code>).
08	Select Values	Renombra columnas, y enriquece con métricas como <code>"es_dia_lluvia"</code> , <code>"categoria_temperatura"</code> , <code>"anio"</code> , <code>"mes"</code> , <code>"dia"</code>
09	Insert/Update	Carga idempotente en

	Staging	staging.observaciones_clean (SCD Tipo 1)
10	Table Input	Lee staging.observaciones_clean ya limpio.
11	Insert/Update DW	Carga final en “olap_snow.dim_clima”, con clave única (fecha, ciudad)

Tras algunas correcciones y ejecuciones de SQL, he conseguido que el flujo se ejecute, como se puede ver en esta captura:



Los datos quedan correctamente escritos, verificado con “SELECT COUNT(*) FROM staging.observaciones_clean” para el Staging y “SELECT COUNT(*) FROM olap_snow.dim_clima” para el OLAP:

```
agroceuta_dw=# SELECT COUNT(*) FROM staging.observaciones_clean;
count
-----
10001
(1 row)

agroceuta_dw=# SELECT COUNT(*) FROM olap_snow.dim_clima;
count
-----
10001
(1 row)
```

3.2. Diseño del flujo ETL para AgroCeuta

3.2.1. Preparación para simular el proceso ETL

Dado que los datos introducidos en la práctica 1 están prácticamente limpios, vamos a alterar un poco el DW de AgroCeuta para implementar un proceso ETL. Para ello, seguiré los siguientes pasos:

1. Insertar datos sucios en el OLTP:

-- 1. Eliminar la constraint NOT NULL de la columna email

```
ALTER TABLE oltp_ventas.clientes ALTER COLUMN email DROP NOT NULL;
```

-- Emails nulos

```
UPDATE oltp_ventas.clientes SET email = NULL WHERE cliente_id % 7 = 0;
```

-- Emails sin '@' (formato inválido)

```
UPDATE oltp_ventas.clientes SET email = 'sincorreio' WHERE cliente_id % 11 = 0;
```

-- Nombres en MAYÚSCULAS (inconsistencia de formato)

```
UPDATE oltp_ventas.clientes SET nombre_cliente = UPPER(nombre_cliente) WHERE cliente_id % 5 = 0;
```

2. Crear las tablas de staging:

```
CREATE SCHEMA IF NOT EXISTS agroceuta_staging;
```

```
CREATE TABLE IF NOT EXISTS agroceuta_staging.stg_clientes_clean (
  clienteid INT PRIMARY KEY,
  nombrecliente VARCHAR(150),
  tipocliente VARCHAR(50),
  canal VARCHAR(50),
  segmento VARCHAR(50),
  paiscliente VARCHAR(100),
  ciudadcliente VARCHAR(100),
  email VARCHAR(150),
  telefono VARCHAR(30)
);
```

```
CREATE TABLE IF NOT EXISTS agroceuta_staging.etl_log_clientes_error (
  id SERIAL PRIMARY KEY,
  clienteid INT,
```

```
email    VARCHAR(150),
motivo   VARCHAR(100),
ts_carga TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
agroceuta_dw=# -- 1. Eliminar la constraint NOT NULL de la columna email
ALTER TABLE oltp_ventas.clientes ALTER COLUMN email DROP NOT NULL;

-- Emails nulos
UPDATE oltp_ventas.clientes SET email = NULL WHERE cliente_id % 7 = 0;

-- Emails sin '@' (formato inválido)
UPDATE oltp_ventas.clientes SET email = 'sincorreio' WHERE cliente_id % 11 = 0;

-- Nombres en MAYÚSCULAS (inconsistencia de formato)
UPDATE oltp_ventas.clientes SET nombre_cliente = UPPER(nombre_cliente) WHERE cliente_id % 5 = 0;
ALTER TABLE
UPDATE 8
UPDATE 5
UPDATE 12
agroceuta_dw=# CREATE SCHEMA IF NOT EXISTS agroceuta_staging;

CREATE TABLE IF NOT EXISTS agroceuta_staging.stg_clientes_clean (
  clienteid      INT PRIMARY KEY,
  nombrecliente  VARCHAR(150),
  tipocliente    VARCHAR(50),
  canal          VARCHAR(50),
  segmento       VARCHAR(50),
  paiscliente    VARCHAR(100),
  ciudadcliente  VARCHAR(100),
  email          VARCHAR(150),
  telefono       VARCHAR(30)
);

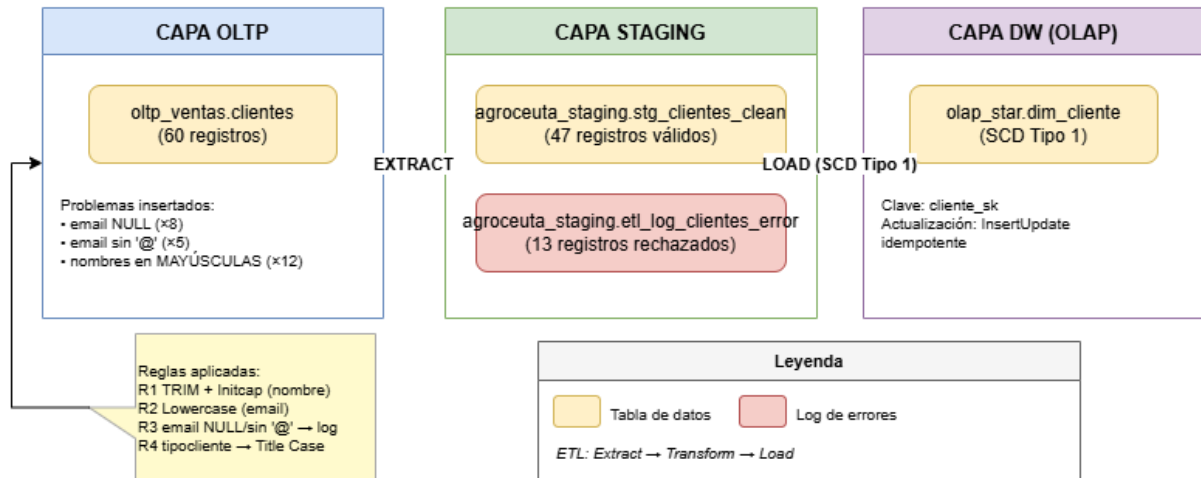
CREATE TABLE IF NOT EXISTS agroceuta_staging.etl_log_clientes_error (
  id              SERIAL PRIMARY KEY,
  clienteid       INT,
  email           VARCHAR(150),
  motivo         VARCHAR(100),
  ts_carga        TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
NOTICE: schema "agroceuta_staging" already exists, skipping
CREATE SCHEMA
CREATE TABLE
NOTICE: relation "etl_log_clientes_error" already exists, skipping
CREATE TABLE
agroceuta_dw=#
```

Con los cambios introducidos, voy a mostrar el diagrama de arquitectura ETL propuesto, así como su flujo de transformación.

3.2.2 Diagrama de Arquitectura ETL y flujo de transformación

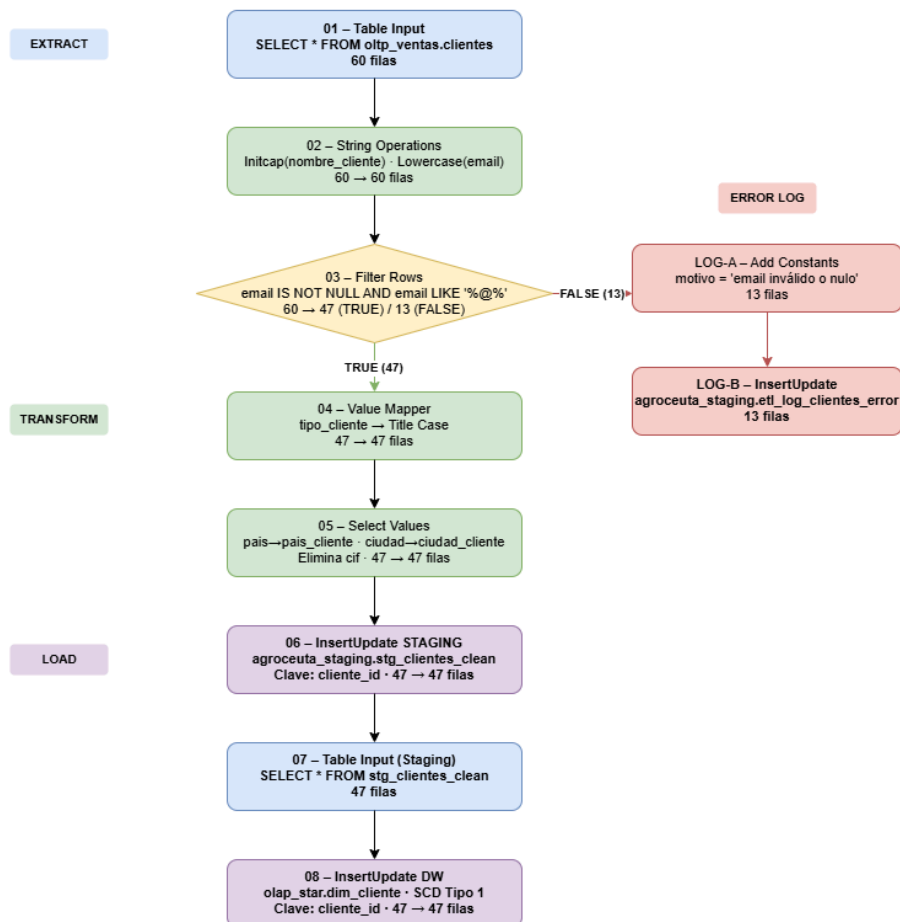
Diagrama de Arquitectura

Arquitectura ETL – AgroCeuta (3 Capas)



Flujo de transformación

Flujo de Transformación – etl_agroceuta.ktr



3.2.3 Justificaciones de diseño

- ¿Por qué ETL y no ELT?

AgroCeuta trabaja con un Data Warehouse relacional en PostgreSQL. La estrategia ETL (Extract-Transform-Load) es la más adecuada porque permite limpiar y validar los datos antes de cargarlos en el DW, garantizando que solo entran datos de calidad. La alternativa ELT (Extract-Load-Transform) es propia de entornos cloud con Data Lakes (BigQuery, Snowflake, Spark), donde el motor de procesamiento reside en la nube y tiene capacidad de transformar datos masivos ya almacenados. En este caso, el volumen es reducido y el DW es relacional-analítico, por lo que ETL con Pentaho PDI es la opción correcta.

- ¿Carga incremental o full load?

En esta práctica se realiza una lectura completa del origen en cada ejecución, pero la carga sobre dimcliente se implementa como Insert/Update SCD Tipo 1 y no como full load físico, para preservar las claves subrogadas ya referenciadas por la tabla de hechos.

4. Implementación del componente ETL con Pentaho

4.1 Catálogo de transformaciones

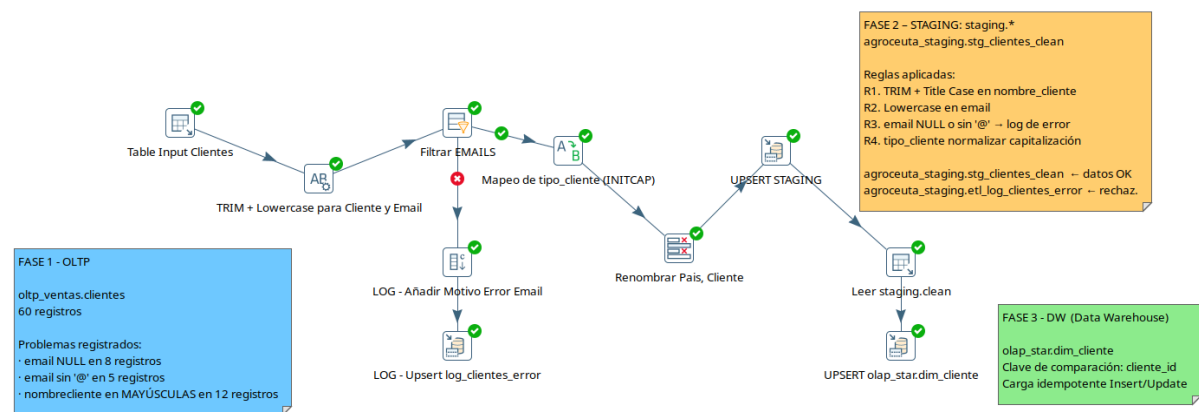
Fichero .ktr	Propósito	Entrada	Salida
limpieza_meteo_completo.ktr	Limpieza y carga del dataset meteorológico	meteo.observaciones_diarias	staging.observaciones_clean → olap_snow.dim_clima
etl_agroceuta.ktr	Limpieza y carga de clientes AgroCeuta (3 capas)	oltp_ventas.clientes	agroceuta_staging.stg_clientes_clean → olap_star.dim_cliente

4.2 Descripción detallada de cada paso

Paso	Configuración	Filas entrada	Filas salida
01 - Table Input (OLTP)	SELECT cliente_id, nombre_cliente, cif, tipo_cliente, canal, segmento, pais, ciudad, email, telefono FROM oltp_ventas.clien tes	-	60
02 - String Operations	Initcap en nombre_cliente, Lowercase en email	60	60
03 - Filter Rows	email IS NOT NULL AND email contains '@'	60	47 (TRUE) / 13 (FALSE)
LOG A - Add Constants	motivo = "email inválido o nulo"	13	13
LOG B - Insert/Update	agroceuta_stagi ng.etl_log_client es_error	13	13
04 - Value Mapper	"tipo_cliente" normalizado a Title Case	47	47
05 - Select Values	Renombre pais → pais_cliente, ciudad → ciudad_cliente; descarta cif (no presente en OLAP)	47	47
06 - Insert/Update (staging)	agroceuta_stagi ng.stg_clientes_ clean, clave "cliente_id", SCD	47	47

	Tipo 1		
07 - Table Input (staging)	SELECT * FROM agroceuta_staging.stg_clientes_clean	-	47
08 - Insert/Update (DW)	olap_star.dim_cliente, clave "cliente_id", SCD Tipo 1	47	47

Trasladamos toda esta configuración a Pentaho y ejecutamos la transformación:



Este es el flujo final, con las notas añadidas sobre la arquitectura definida previamente.

IMPORTANTE: Hay que ejecutar dos veces la transformación, una primera para que se carguen los datos en el staging_clean; y otra para cargarlos al OLAP. En caso de haber errores durante la ejecución, se pueden romper el salto (hop) entre los pasos 6 y 7.

Y estos son los resultados sobre el DW:

Execution Results													
Logging Execution History Step Metrics Performance Graph Metrics Preview data													
Nombre paso	Numero Copia	Leído	Escrito	Entrada	Salida	Actualizado	Rejected	Errores	Activo	Tiempo	Velocidad (r/s)	PrivE/S	
1 Table Input Clientes	0	0	60	60	0	0	0	0	Finalizado	0.0s	1.622		
2 TRIM + Lowercase para Cliente y Email	0	60	60	0	0	0	0	0	Finalizado	0.0s	1.538		
3 Filtrar EMAILS	0	60	60	0	0	0	0	0	Finalizado	0.0s	1.395		
4 Mapeo de tipo_cliente (INITCAP)	0	47	47	0	0	0	0	0	Finalizado	0.0s	1.022		
5 LOG - Añadir Motivo Error Email	0	13	13	0	0	0	0	0	Finalizado	0.0s	283		
6 LOG - Upsert log_clientes_error	0	13	13	13	0	0	0	0	Finalizado	0.1s	224		
7 Renombrar Pais, Cliente	0	47	47	0	0	0	0	0	Finalizado	0.0s	959		
8 UPSERT STAGING	0	47	47	47	0	0	0	0	Finalizado	0.1s	595		
9 Leer staging.clean	0	0	47	47	0	0	0	0	Finalizado	0.0s	1.119		
10 UPSERT olap_star.dim_cliente	0	47	47	47	0	0	0	0	Finalizado	0.1s	618		

Luego, para verificar estos resultados, he ejecutado unas consultas en la base de datos para verificar que los resultados concuerdan:

```
agroceuta_dw=# -- Staging poblado correctamente
SELECT COUNT(*) FROM agroceuta_staging.stg_clientes_clean;
-- Esperado: ~47 (los 60 - los rechazados por email)

-- Log de errores
SELECT COUNT(*) FROM agroceuta_staging.etl_log_clientes_error;
-- Esperado: ~13

-- DW cargado desde staging
SELECT COUNT(*) FROM olap_star.dim_cliente;
count
-----
47
(1 row)

count
-----
13
(1 row)

count
-----
60
(1 row)
agroceuta_dw=#
```

He comprobado que el staging se ha cargado y transformado correctamente; el log tiene las tuplas erróneas; y el DW se actualiza, pero mantiene todos los registros por los siguientes motivos:

- Aunque inicialmente se planteó como una estrategia de full load por el reducido tamaño de la dimensión de clientes, la implementación final no se ha materializado como un truncate-load físico del destino, sino como una carga completa del conjunto fuente con actualización idempotente mediante InsertUpdate y SCD Tipo 1. En cada ejecución se leen los 60 registros de oltpventas.clientes, se aplican las reglas de calidad definidas y únicamente los 47 registros válidos continúan hacia olapstar.dimcliente, mientras que los 13 registros con email inválido o nulo se desvían al log de errores.
- Esta decisión es coherente con el diseño del Data Warehouse, ya que olap_star.dim_cliente utiliza una clave subrogada (clientes_k) y la tabla de hechos olap_star.fact_exportaciones referencia dicha clave mediante una FK. Por ello, truncar y recargar la dimensión una vez pobladas las facts podría alterar los identificadores subrogados y comprometer la integridad referencial del modelo analítico, salvo que se reconstruyeran también las tablas de hechos dependientes.
- Por ello, la estrategia aplicada puede describirse con mayor precisión como una recarga completa del origen con validación previa y Insert/Update sobre la dimensión destino. Este enfoque mantiene la idempotencia del proceso, preserva las claves analíticas existentes y resulta adecuado para una dimensión pequeña donde interesa corregir atributos válidos sin romper la relación con las tablas de hechos ya cargadas.

5. Integración de objetivos en consulta analítica

5.1 Actualización del catálogo de transformaciones

El catálogo de transformaciones incluye ahora un nuevo fichero .ktr, utilizado para realizar el Hito 4 de esta práctica. El catálogo resultante es el siguiente:

Fichero .ktr	Propósito	Entrada	Salida
limpieza_meteo_completo.ktr	Limpieza y carga del dataset meteorológico	meteo.observaciones_diarias	staging.observaciones_clean → olap_snow.dim_clima
etl_agroceuta.ktr	Limpieza y carga de clientes AgroCeuta (3 capas)	oltp_ventas.clientes	agroceuta_staging.stg_clientes_clean → olap_star.dim_cliente
analisis_olap_agroceuta.ktr	Análisis OLAP: ingresos vs objetivos por familia	olap_star.fact_exportaciones + objetivos_familias.csv	staging.resultados_analisis + analisis_ventas_agroceuta.csv

5.2 Flujo de transformación propuesto

Paso	Configuración	Filas entrada	Filas salida
01 - Table Input	<pre>SELECT dp.familia_producto AS familia, SUM(f.importe_netto) AS ingresos_reales, SUM(f.coste_total) AS costes_totales, SUM(f.margen) AS margen_total FROM olap_star.fact_exportaciones f JOIN olap_star.dim_producto dp ON dp.producto_sk = f.producto_sk</pre>	-	4

	GROUP BY dp.familia_producto ORDER BY dp.familia_producto		
02 - CSV File Input	"objetivos_familias.csv", separador ";"	-	4
03 - Stream Lookup	Clave: "familia"; recupera "objetivo_ingresos" del CSV	4	4
04 - Add Constants	CIEN=100 (Number)	4	4
05 - Calculator	4 operaciones: ratio_margen, pct_margen, ratio_objetivo y pct_objetivo	4	4
06 - Sort Rows	Ordena por "ingresos_reales" DESC	4	4
07 - Insert/Update	staging.resultados_analisis, clave "familia"	4	4
08 - Text File Output	" analisis_ventas_agroceuta.csv", separador ";", sin campos "ratio_*" ni "CIEN"	4	4

Para el paso 2, este es el CSV que he creado (se adapta a los datos de mi DW):

```
familia_producto;objetivo_ingresos
Citricos;35000
Frutas;45000
Frutos rojos;60000
Hortalizas;85000
```

El paso 7 (almacenamiento en agroceuta_staging.resultados_analisis) implica la creación de una nueva tabla. El Insert/Update se configura como en la siguiente captura:

Nombre de paso

Conexión: PracticasSM

Esquema destino: agroceuta_staging

Tabla destino: resultados_analisis

Tamaño de transacción (commit): 100

No realizar actualizaciones: ☐

La clave(s) para realizar búsqueda de valor(es):

^	Campo de tabla	Comparador	Campo1	Campo2	Obtener Campos
1	familia_producto	=	familia_producto		

Campos de actualización:

^	Campo de tabla	Campo de Flujo	Actualizar	Obtener campos de actualización
1	familia_producto	familia_producto	N	
2	ingresos_reales	ingresos_reales	Y	
3	costes_totales	costes_totales	Y	
4	margen_total	margen_total	Y	
5	objetivo	objetivo	Y	
6	pct_margen	pct_margen	Y	
7	pct_objetivo	pct_objetivo	Y	

Edit mapping

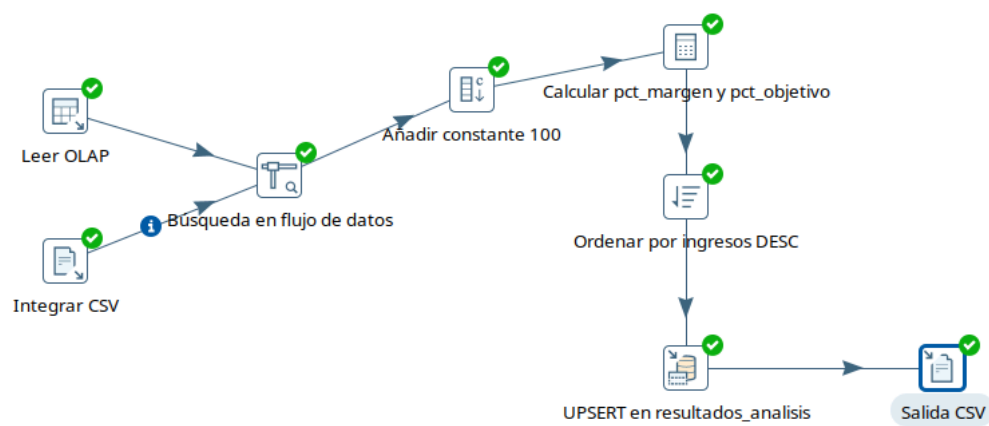
Con esta configuración, deberemos hacer click en “SQL” y Ejecutar todos los comandos generados para crear la nueva tabla en el staging.

Tras hacer click en SQL, debería verse algo similar a esto:

```
CREATE TABLE agroceuta_staging.resultados_analisis
(
  familia_producto VARCHAR(100)
, ingresos_reales DOUBLE PRECISION
, costes_totales DOUBLE PRECISION
, margen_total DOUBLE PRECISION
, objetivo BIGINT
, pct_margen DOUBLE PRECISION
, pct_objetivo DOUBLE PRECISION
)
;CREATE INDEX idx_resultados_analisis_lookup ON
agroceuta_staging.resultados_analisis(familia_producto)
;
```

Si coincide, podemos ejecutar el SQL y proceder con la transformación.

Ahora, traslado toda esta configuración a Pentaho. Este es el flujo final:



Y estos son los resultados de la ejecución de la transformación:

etl_agroceuta analisis_olap_agroceuta

Execution Results

Logging Execution History Step Metrics Performance Graph Metrics Preview data

Nombre paso	Numero Copia	Leído	Escrito	Entrada	Salida	Actualizado	Rejected	Errores	Activo	Tiempo	Velocidad (r/s)	Priv/S
1 Leer OLAP	0	0	4	4	0	0	0	0	Finalizado	0.0s	121	-
2 Integrar CSV	0	0	4	5	0	0	0	0	Finalizado	0.0s	1,667	-
3 Búsqueda en flujo de datos	0	8	4	0	0	0	0	0	Finalizado	0.2s	51	-
4 Añadir constante 100	0	4	4	0	0	0	0	0	Finalizado	0.2s	25	-
5 Calcular pct_margen y pct_objetivo	0	4	4	0	0	0	0	0	Finalizado	0.2s	25	-
6 Ordenar por ingresos DESC	0	4	4	0	0	0	0	0	Finalizado	0.2s	24	-
7 UPSERT en resultados_analisis	0	4	4	4	0	0	0	0	Finalizado	0.2s	22	-
8 Salida CSV	0	4	4	0	5	0	0	0	Finalizado	0.2s	27	-

Vemos que todo se ejecuta correctamente.

5.3. Resultado final de la transformación

Esto es si lo vemos desde PostgreSQL:

```

agroceuta_dw=# SELECT * FROM agroceuta_staging.resultados_analisis ORDER BY ingresos_reales DESC;
 familia_producto | ingresos_reales | costes_totales | margen_total | objetivo | pct_margen | pct_objetivo
-----+-----+-----+-----+-----+-----+-----
 Hortalizas       | 47250.78        | 13627.31       | 33623.47     | 85000    | 71.15960837048615 | 55.589152941176465
 Frutos rojos     | 41898.54        | 6681.81        | 35216.73     | 60000    | 84.05240373530916 | 69.8309
 Citricos         | 15171.72        | 4627.1         | 10544.62     | 35000    | 69.50180994640027 | 43.34777142857143
 Frutas           | 14037.81        | 1940.94        | 12096.87     | 45000    | 86.1734843255465  | 31.195133333333334
(4 rows)

agroceuta_dw=#

```

Este es el CSV generado por el .ktr:

```

1  familia_producto;ingresos_reales;costes_totales;margen_total;objetivo;pct_margen;pct_objetivo
2  Hortalizas;47250.78;13627.31;33623.47;85000;71,1596083705;55,5891529412
3  Frutos rojos;41898.54;6681.81;35216.73;60000;84,0524037353;69,8309
4  Citricos;15171.72;4627.1;10544.62;35000;69,5018099464;43,3477714286
5  Frutas;14037.81;1940.94;12096.87;45000;86,1734843255;31,1951333333
6

```

Anexo A. Archivos adjuntos entregados

Junto con este documento se adjunta un enlace a un repositorio GitHub que contiene, además de todo lo realizado en la práctica anterior (1), todo lo relacionado con la Práctica 2:

- Diagrama de Arquitectura ETL
- Flujo de transformación
- Los archivos .ktr asociados a las transformaciones realizadas.
- El CSV creado para el hito 4

Enlace al repositorio de GitHub: <https://github.com/patxisantos/sm-pl>